Graphs Lab

Data Structures & Algorithms

Lab 09 -- Building Graphs and Traversing Them

Duration: 2 hours (2 challenges)

Lab Structure

Each challenge follows this format:

1. **Challenge Presentation** (5 min)
2. **Your Implementation Time** (25 min)
3. **Pseudocode Review** (5 min)
4. **Solution Walkthrough** (10 min)
5. **Code Explanation** (15 min)

Total per challenge: ~60 minutes

Today's Challenges

Challenge 1: Campus Map -- Building a Graph and BFS

- Represent a small campus as an **adjacency list graph**
- Use **Breadth-First Search (BFS)** to find the fewest-hop path from the main gate to every building
- Understand why BFS always finds the shortest path in an unweighted graph

Challenge 2: Social Network -- DFS and Connected Components

- Build a friendship network as a graph where some users are not connected to others
- Use **Depth-First Search (DFS)** to explore all friends reachable from a given person
- Count how many **separate friend groups** (connected components) exist in the network

Quick Recap: What is a Graph?

A **graph** $G = (V, E)$ consists of a set of **vertices** (nodes) and a set of **edges** (connections between nodes).

```
Undirected edge:   A --- B    (relationship goes both ways)
Directed edge:     A --> B    (relationship goes one way only)
```

Two Common Representations

Adjacency Matrix: A 2D array where `matrix[i][j] = 1` if there is an edge from vertex `i` to vertex `j`.

Adjacency List: An array of lists where `list[i]` contains all neighbors of vertex `i`.

We use **adjacency lists** in today's lab -- they use far less memory when the graph is sparse (few edges compared to vertices), which is the common case in real-world graphs.

Quick Recap: BFS vs DFS

Property	BFS	DFS
Data structure	Queue (FIFO)	Stack (LIFO) or Recursion
Explores	Level by level (nearest first)	As deep as possible first
Finds shortest path?	Yes (unweighted graphs)	Not necessarily
Use case	Shortest path, level-order	Reachability, components
Time complexity	O(V + E)	O(V + E)

Both visit every vertex and edge exactly once -- they differ only in the **order** of exploration.

Challenge 1

Campus Map -- Building a Graph and BFS

Challenge 1: Problem Statement

Scenario: Your college campus has 6 key locations connected by walking paths:

```
0 = Main Gate    1 = Library     2 = Cafeteria
3 = Computer Lab 4 = Gymnasium   5 = Hostel
```

Paths (undirected edges):

- Gate -- Library, Gate -- Cafeteria
- Library -- Lab, Library -- Gym
- Cafeteria -- Gym
- Lab -- Hostel, Gym -- Hostel

Tasks:

1. Build this graph using an **adjacency list**
2. Display the adjacency list
3. Run **BFS starting from the Main Gate (vertex 0)**
4. Print the BFS traversal order and the **distance** (hops) from the Gate to each location

Time: 25 minutes

Challenge 1: Think About It First

Draw the graph and trace BFS by hand before coding.

```
         [0 Gate]
         /       \
   [1 Library] [2 Cafeteria]
    /      \         \
[3 Lab]  [4 Gym]---[4 Gym]
    \      /
  [5 Hostel]
```

```
Adjacency List:
  Gate(0):      Library(1), Cafeteria(2)
  Library(1):   Gate(0), Lab(3), Gym(4)
  Cafeteria(2): Gate(0), Gym(4)
  Lab(3):       Library(1), Hostel(5)
  Gym(4):       Library(1), Cafeteria(2), Hostel(5)
  Hostel(5):    Lab(3), Gym(4)
```

BFS starts at Gate. It visits all distance-1 neighbors before any distance-2 neighbors, exactly like ripples spreading out in water.

Challenge 1: BFS Trace

```
Initial: Queue = [Gate(0)],  Visited = {0},  dist[0] = 0

Step 1: Dequeue Gate(0).
  Unvisited neighbors: Library(1), Cafeteria(2)
  Enqueue both. dist[1] = 1, dist[2] = 1.
  Queue = [Library(1), Cafeteria(2)]

Step 2: Dequeue Library(1).
  Unvisited neighbors: Lab(3), Gym(4)
  Enqueue both. dist[3] = 2, dist[4] = 2.
  Queue = [Cafeteria(2), Lab(3), Gym(4)]

Step 3: Dequeue Cafeteria(2).
  Unvisited neighbors: none (Gym already enqueued).
  Queue = [Lab(3), Gym(4)]

Step 4: Dequeue Lab(3). Enqueue Hostel(5). dist[5] = 3.
Step 5: Dequeue Gym(4). Hostel already visited.
Step 6: Dequeue Hostel(5). No new neighbors.
```

BFS Order: Gate, Library, Cafeteria, Lab, Gym, Hostel

Distances from Gate: Library=1, Cafeteria=1, Lab=2, Gym=2, Hostel=3

Challenge 1: Pseudocode

```
ALGORITHM CampusMapBFS

  Data: adjacency list adj[], queue Q, visited[], dist[]

  buildGraph():
    - addEdge(u, v): append v to adj[u], append u to adj[v]
    - Add all 7 edges described in the problem

  printAdjList():
    - For each vertex v, print adj[v]

  BFS(start):
    - Mark all vertices unvisited, set all dist[] = -1
    - dist[start] = 0, enqueue start, mark start visited
    - While Q is not empty:
      - u = dequeue from Q
      - Print u
      - For each neighbor v of u:
        - If v is not visited:
          - Mark v visited
          - dist[v] = dist[u] + 1
          - Enqueue v
    - Print distances to all vertices

END ALGORITHM
```

Challenge 1: Solution Code -- Setup and Graph Building

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define VERTICES 6
#define MAX_EDGES 20

// Adjacency list using arrays (simple version)
int adj[VERTICES][VERTICES];
int degree[VERTICES];  // How many neighbors each vertex has

const char* name[] = {"Gate", "Library", "Cafeteria", "Lab", "Gym", "Hostel"};

void addEdge(int u, int v) {
    adj[u][degree[u]++] = v;  // Add v to u's list
    adj[v][degree[v]++] = u;  // Add u to v's list (undirected)
}

void buildGraph() {
    memset(degree, 0, sizeof(degree));
    addEdge(0, 1);  // Gate -- Library
    addEdge(0, 2);  // Gate -- Cafeteria
    addEdge(1, 3);  // Library -- Lab
    addEdge(1, 4);  // Library -- Gym
    addEdge(2, 4);  // Cafeteria -- Gym
    addEdge(3, 5);  // Lab -- Hostel
    addEdge(4, 5);  // Gym -- Hostel
}
```

Challenge 1: Solution Code -- Print and BFS

```
void printAdjList() {
    printf("--- Adjacency List ---\n");
    for (int u = 0; u < VERTICES; u++) {
        printf("  %s(%d): ", name[u], u);
        for (int i = 0; i < degree[u]; i++) {
            int v = adj[u][i];
            printf("%s(%d)", name[v], v);
            if (i < degree[u] - 1) printf(" --> ");
        }
        printf("\n");
    }
}

void BFS(int start) {
    int visited[VERTICES] = {0};
    int dist[VERTICES];
    int queue[VERTICES], front = 0, rear = 0;

    for (int i = 0; i < VERTICES; i++) dist[i] = -1;

    visited[start] = 1;
    dist[start] = 0;
    queue[rear++] = start;

    printf("\n--- BFS from %s ---\n  Order: ", name[start]);
    while (front < rear) {
        int u = queue[front++];
        printf("%s", name[u]);
        for (int i = 0; i < degree[u]; i++) {
            int v = adj[u][i];
            if (!visited[v]) {
                visited[v] = 1;
                dist[v] = dist[u] + 1;
                queue[rear++] = v;
            }
        }
        if (front < rear) printf(" --> ");
    }
    printf("\n\n--- Distance from Gate ---\n");
    for (int i = 0; i < VERTICES; i++)
        printf("  %s: %d hop%s\n", name[i], dist[i], dist[i] == 1 ? "" : "s");
}
```

Challenge 1: Solution Code -- Main

```
int main() {
    printf("=== Challenge 1: Campus Map (Adjacency List + BFS) ===\n\n");

    buildGraph();
    printAdjList();
    BFS(0);  // Start BFS from Main Gate (vertex 0)

    return 0;
}
```

Challenge 1: Expected Output

```
=== Challenge 1: Campus Map (Adjacency List + BFS) ===

--- Adjacency List ---
  Gate(0):      Library(1) --> Cafeteria(2)
  Library(1):   Gate(0) --> Lab(3) --> Gym(4)
  Cafeteria(2): Gate(0) --> Gym(4)
  Lab(3):       Library(1) --> Hostel(5)
  Gym(4):       Library(1) --> Cafeteria(2) --> Hostel(5)
  Hostel(5):    Lab(3) --> Gym(4)

--- BFS from Gate ---
  Order: Gate --> Library --> Cafeteria --> Lab --> Gym --> Hostel

--- Distance from Gate ---
  Gate:      0 hops
  Library:   1 hop
  Cafeteria: 1 hop
  Lab:       2 hops
  Gym:       2 hops
  Hostel:    3 hops
```

Observation: Library and Cafeteria are both 1 hop from the Gate (they are directly connected). The Hostel takes 3 hops -- the longest journey on this campus. BFS guarantees these are the minimum possible.

Challenge 1: Code Explanation

Part 1: The Adjacency List Structure

```
int adj[VERTICES][VERTICES];
int degree[VERTICES];
```

We use a 2D array where `adj[u]` is the list of neighbors for vertex `u`, and `degree[u]` tracks how many neighbors `u` has.

```
After buildGraph():
  adj[0] = [1, 2]       degree[0] = 2
  adj[1] = [0, 3, 4]    degree[1] = 3
  adj[2] = [0, 4]       degree[2] = 2
  adj[3] = [1, 5]       degree[3] = 2
  adj[4] = [1, 2, 5]    degree[4] = 3
  adj[5] = [3, 4]       degree[5] = 2
```

Each `addEdge(u, v)` call adds `v` to u's list AND `u` to v's list -- this is what makes the graph undirected. One function call creates two entries.

Challenge 1: Code Explanation

Part 2: How BFS Uses a Queue

The queue is the heart of BFS. Think of it as a waiting line:

```
Initial:
  Queue: [Gate]    dist[Gate] = 0

After processing Gate (dequeue, enqueue neighbors):
  Queue: [Library, Cafeteria]    dist[Library] = 1, dist[Cafeteria] = 1

After processing Library:
  Queue: [Cafeteria, Lab, Gym]   dist[Lab] = 2, dist[Gym] = 2

After processing Cafeteria (Gym already visited -- skip):
  Queue: [Lab, Gym]

...and so on
```

The key: We only enqueue a vertex when we first see it (`if (!visited[v])`). The first time BFS reaches a vertex is always via the shortest path, because BFS processes all distance-k vertices before any distance-$(k+1)$ vertex.

Challenge 1: Code Explanation

Part 3: Why BFS Finds Shortest Paths

BFS processes vertices in "waves" -- all vertices at distance 1 before distance 2, all at distance 2 before distance 3, and so on.

```
Wave 0 (distance 0):  Gate
Wave 1 (distance 1):  Library, Cafeteria
Wave 2 (distance 2):  Lab, Gym
Wave 3 (distance 3):  Hostel
```

When we first encounter a vertex `v` while processing vertex `u`, we set `dist[v] = dist[u] + 1`. Since `u` is at the smallest current distance, this is the shortest distance to `v`. Any later path to `v` would go through a vertex at the same or greater distance -- it cannot be shorter.

This is why BFS is the go-to algorithm for shortest path in unweighted graphs. Weighted shortest paths (with different edge costs) require Dijkstra's algorithm instead.

Challenge 2

Social Network -- DFS and Connected Components

Challenge 2: Problem Statement

Scenario: You are analyzing a social network with 7 users. Some users are friends with each other, but the network has **separate friend groups** that are not connected to each other.

```
Users: 0=Alice  1=Bob  2=Carol  3=Dave  4=Eve  5=Frank  6=Gina
```

Friendships (undirected edges):

- Alice--Bob, Alice--Carol, Bob--Dave, Carol--Dave
- Eve--Frank
- Gina has no friends (isolated)

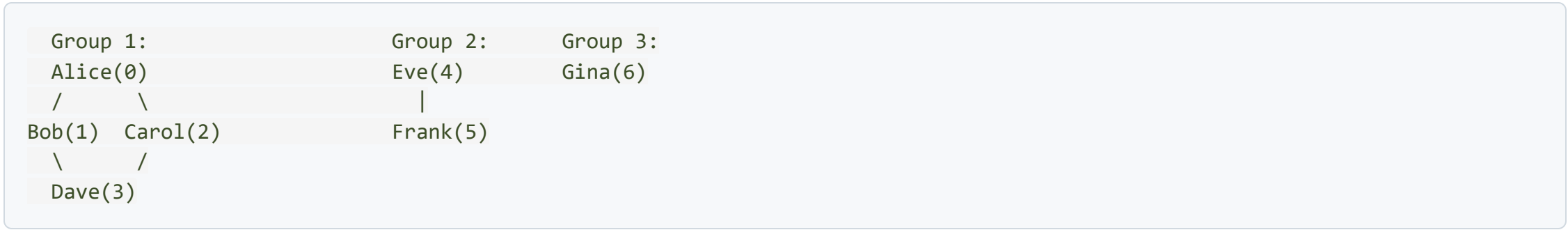
Tasks:

1. Build the friendship graph using an adjacency list
2. Run **DFS from Alice** and print every friend reachable from her
3. Find and print **all connected components** (separate friend groups) in the network
4. Count the total number of groups

Time: 25 minutes

Challenge 2: Think About It First

Sketch the graph -- notice the disconnected structure:



What DFS from Alice will find:

- Visit Alice(0), go deep to Bob(1), then Dave(3), then Carol(2)
- DFS never reaches Eve, Frank, or Gina -- they are in different components

Expected connected components:

- Component 1: {Alice, Bob, Carol, Dave} -- 4 users
- Component 2: {Eve, Frank} -- 2 users
- Component 3: {Gina} -- 1 user (isolated)

To find ALL components, we run DFS once per unvisited vertex after the first DFS finishes.

Challenge 2: DFS Trace (from Alice)

```
DFS(Alice=0):
  Visit Alice(0). Neighbors: Bob(1), Carol(2).
    DFS(Bob=1):
      Visit Bob(1). Neighbors: Alice(0)[visited], Dave(3).
        DFS(Dave=3):
          Visit Dave(3). Neighbors: Bob(1)[visited], Carol(2).
            DFS(Carol=2):
              Visit Carol(2). Neighbors: Alice(0)[visited], Dave(3)[visited].
              No unvisited neighbors. Backtrack to Dave.
            Backtrack to Bob.
          Backtrack to Alice.
      Next neighbor of Alice: Carol(2) -- already visited.
  Done.
```

DFS order from Alice: Alice, Bob, Dave, Carol

Notice: DFS went as deep as possible (Alice -> Bob -> Dave -> Carol) before backtracking. This is the opposite of BFS which would have visited Bob and Carol first (both are 1 hop from Alice).

Challenge 2: Pseudocode

```
ALGORITHM SocialNetworkDFS

  Data: adjacency list adj[], visited[], componentId[]

  DFS(u, compId):
    - Mark u as visited
    - componentId[u] = compId
    - Print u
    - For each neighbor v of u:
      - If v is not visited: DFS(v, compId)

  findAllComponents():
    - Mark all vertices unvisited
    - compId = 0
    - For each vertex u from 0 to V-1:
      - If u is not visited:
          compId++
          Print "Component compId:"
          DFS(u, compId)
    - Print total number of components

  main():
    - buildGraph() with 7 vertices and friendships
    - Print adjacency list
    - Run DFS from Alice (vertex 0) only
    - Run findAllComponents()

END ALGORITHM
```

Challenge 2: Solution Code -- Setup and Graph

```c
#include <stdio.h>
#include <string.h>

#define VERTICES 7
#define MAX_DEG  10

int adj[VERTICES][MAX_DEG];
int degree[VERTICES];
int visited[VERTICES];
int componentId[VERTICES];

const char* uname[] = {
    "Alice", "Bob", "Carol", "Dave", "Eve", "Frank", "Gina"
};

void addEdge(int u, int v) {
    adj[u][degree[u]++] = v;
    adj[v][degree[v]++] = u;
}

void buildGraph() {
    memset(degree, 0, sizeof(degree));
    addEdge(0, 1);  // Alice -- Bob
    addEdge(0, 2);  // Alice -- Carol
    addEdge(1, 3);  // Bob -- Dave
    addEdge(2, 3);  // Carol -- Dave
    addEdge(4, 5);  // Eve -- Frank
    // Gina (6) has no edges -- isolated node
}
```

Challenge 2: Solution Code -- DFS and Print Adjacency List

```
void printAdjList() {
    printf("--- Friendship Graph (Adjacency List) ---\n");
    for (int u = 0; u < VERTICES; u++) {
        printf("  %s(%d): ", uname[u], u);
        if (degree[u] == 0) {
            printf("(no friends)\n");
            continue;
        }
        for (int i = 0; i < degree[u]; i++) {
            int v = adj[u][i];
            printf("%s(%d)", uname[v], v);
            if (i < degree[u] - 1) printf(" --> ");
        }
        printf("\n");
    }
}

void DFS(int u, int compId) {
    visited[u] = 1;
    componentId[u] = compId;
    printf("  %s", uname[u]);

    for (int i = 0; i < degree[u]; i++) {
        int v = adj[u][i];
        if (!visited[v]) {
            printf(" --[visits]--> ");
            DFS(v, compId);          // Recursive call: go deeper
        }
    }
}
```

Challenge 2: Solution Code -- Find All Components and Main

```
void findAllComponents() {
    memset(visited, 0, sizeof(visited));
    int totalComp = 0;

    printf("\n--- Connected Components (Friend Groups) ---\n");
    for (int u = 0; u < VERTICES; u++) {
        if (!visited[u]) {
            totalComp++;
            printf("\n  Group %d: ", totalComp);
            DFS(u, totalComp);
            printf("\n");
        }
    }
    printf("\n  Total separate friend groups: %d\n", totalComp);
}

int main() {
    printf("=== Challenge 2: Social Network (DFS + Components) ===\n\n");

    buildGraph();
    printAdjList();

    printf("\n--- DFS from Alice only ---\n  Path: ");
    memset(visited, 0, sizeof(visited));
    DFS(0, 1);
    printf("\n");
    printf("  (Eve, Frank, Gina are unreachable from Alice)\n");

    findAllComponents();

    return 0;
}
```

Challenge 2: Expected Output

```
=== Challenge 2: Social Network (DFS + Components) ===

--- Friendship Graph (Adjacency List) ---
  Alice(0): Bob(1) --> Carol(2)
  Bob(1):   Alice(0) --> Dave(3)
  Carol(2): Alice(0) --> Dave(3)
  Dave(3):  Bob(1) --> Carol(2)
  Eve(4):   Frank(5)
  Frank(5): Eve(4)
  Gina(6):  (no friends)

--- DFS from Alice only ---
  Path: Alice --[visits]--> Bob --[visits]--> Dave --[visits]--> Carol
  (Eve, Frank, Gina are unreachable from Alice)
```

Challenge 2: Expected Output (continued)

```
--- Connected Components (Friend Groups) ---

  Group 1: Alice --[visits]--> Bob --[visits]--> Dave
                --[visits]--> Carol

  Group 2: Eve --[visits]--> Frank

  Group 3: Gina

  Total separate friend groups: 3
```

What happened?

- Group 1 contains Alice, Bob, Dave, Carol -- all reachable from Alice via DFS.
- Group 2 contains Eve and Frank -- they only know each other.
- Group 3 is just Gina -- she has no friends at all (degree 0).

A social platform could use this algorithm to show users: "You are in a network of 4 people" and suggest that users in Group 2 might want to connect with users in Group 1.

Challenge 2: Code Explanation

Part 1: How Recursive DFS Works

Each call to `DFS(u)` does three things:

```
DFS(Alice=0):
  1. Mark Alice as visited
  2. Print "Alice"
  3. For each neighbor (Bob, Carol):
       If Bob not visited -> call DFS(Bob)

DFS(Bob=1):
  1. Mark Bob as visited
  2. Print "Bob"
  3. For each neighbor (Alice, Dave):
       Alice is visited -- skip
       Dave not visited -> call DFS(Dave)

... and so on, going deeper until no unvisited neighbors remain
```

The **call stack** acts as the DFS stack. Each recursive call pushes a new frame; backtracking happens automatically when a call returns -- no explicit stack needed.

Challenge 2: Code Explanation

Part 2: Finding All Connected Components

The key to finding all components is a simple outer loop:

```
for (int u = 0; u < VERTICES; u++) {
    if (!visited[u]) {
        totalComp++;
        DFS(u, totalComp);
    }
}
```

- If DFS from the previous starting vertex already visited `u`, we skip it.
- If `u` is still unvisited, it belongs to a new, separate component.
- We start a new DFS from `u`, which will explore the entire new component.

This guarantees every vertex is visited **exactly once** across all DFS calls. The total time is still $O(V + E)$ -- the outer loop and all inner DFS calls together never re-examine an already-visited vertex.

Challenge 2: Code Explanation

Part 3: DFS vs BFS -- Same Graph, Different Order

Run both on the same graph starting from Alice:

```
BFS from Alice (level-by-level, queue):
  Distance 0: Alice
  Distance 1: Bob, Carol    (both are direct friends of Alice)
  Distance 2: Dave          (friend of Bob and Carol)
  BFS Order: Alice, Bob, Carol, Dave

DFS from Alice (depth-first, recursion):
  Visit Alice -> go deep to Bob -> go deep to Dave
  -> backtrack to Dave, visit Carol
  DFS Order: Alice, Bob, Dave, Carol
```

BFS discovered Bob and Carol at the same level because they are both one hop from Alice.

DFS followed the path Alice -> Bob -> Dave all the way to the bottom before backtracking to Carol.

Both visit all 4 reachable vertices, but in a completely different order.

Challenge 2: Code Explanation

Part 4: Adjacency List vs Matrix -- Which to Use?

Feature	Adjacency Matrix	Adjacency List
Space	O(V^2) -- always	O(V + E) -- depends on edges
Check if edge (u,v) exists	O(1)	O(degree of u)
Find all neighbors of u	O(V) -- scan whole row	O(degree of u)
Best for	Dense graphs (E close to V^2)	Sparse graphs (E much less than V^2)

This social network: 7 users, only 5 friendships. A matrix would be 7x7 = 49 cells, mostly zero. The adjacency list uses only 10 entries (2 per edge). For real social networks with millions of users but sparse friendships, adjacency lists save enormous memory.

Lab Session Complete!

Summary: What You Built

Challenge 1: Campus Map with BFS

- Built a 6-vertex undirected graph using an adjacency list
- Implemented BFS using a queue to explore locations level by level
- Verified that BFS always visits nearest locations first and gives shortest hop-count distances
- Distances found: Library=1, Cafeteria=1, Lab=2, Gym=2, Hostel=3 (from Gate)

Challenge 2: Social Network with DFS and Connected Components

- Built a 7-vertex graph with 3 separate connected components
- Ran DFS recursively from Alice -- reached all 4 users in her group, missed Eve/Frank/Gina
- Used an outer loop over all vertices to discover all 3 components automatically
- Compared DFS and BFS traversal orders on the same graph

Key Concepts Reinforced

Graph Representation

- **Adjacency List:** Array of neighbor lists. Space-efficient for sparse graphs. `addEdge(u, v)` adds to both `adj[u]` and `adj[v]` for undirected graphs.

BFS (Breadth-First Search)

- Uses a **queue**. Processes neighbors before going deeper.
- Guarantees **shortest hop-count path** in unweighted graphs.
- Explores one "wave" (distance level) at a time.

DFS (Depth-First Search)

- Uses **recursion** (implicit call stack). Goes as deep as possible before backtracking.
- Does not guarantee shortest paths, but excellent for **reachability** and **component detection**.

Connected Components

- A connected component is a maximal set of vertices all reachable from each other.
- Found by running DFS/BFS from every unvisited vertex. Total time: O(V + E).

Complexity Summary

Operation	Time	Space	Notes
Build graph (add edge)	O(1) per edge	O(V + E) total	Adjacency list
BFS traversal	O(V + E)	O(V)	Queue + visited array
DFS traversal	O(V + E)	O(V)	Call stack + visited array
Find all components	O(V + E)	O(V)	Outer loop + DFS/BFS

Comparison with other structures:

Structure	Search	Insert	Ordered?	Graph support?
Array / Hash Table	O(1) avg	O(1) avg	No	No
BST	O(log n)	O(log n)	Yes	No
Graph (adj list)	O(V + E)	O(1)	No	Yes -- models relationships

Graphs are not faster than arrays for simple lookups -- they are the right tool when **relationships between elements** are what matters.

Take-Home Challenge (Optional)

Extend your implementations with these exercises:

1. **Directed Graph:** Modify Challenge 1 so edges are one-way (directed). For example, you can walk from the Gate to the Library, but there is a shortcut making the Library-to-Gate path different. Change `addEdge` to only add `v` to `adj[u]` (not the reverse). How does BFS change? Can you still reach every location from the Gate?
2. **Path Reconstruction:** In Challenge 1, BFS finds the shortest distance to each location. Modify BFS to also record the `parent[]` array -- when we first visit vertex `v` from vertex `u`, set `parent[v] = u`. After BFS, use the parent array to reconstruct and print the actual shortest path from the Gate to the Hostel (not just the distance).
3. **Think about it:** In Challenge 2, what would happen if Eve and Bob became friends? How many connected components would there be? Draw the new adjacency list and trace how `findAllComponents` would change. What is the maximum number of edges a graph with 7 vertices can have while still being connected?